

Home Assignment:

Identify a real-time office use case that solves a common operational problem.

Do not choose the “Test Case Generator” use case, as it has already been completed. Try to identify and finalize a different unique problem statement.

Once finalized, design and prepare a complete architecture document for the solution.

Note : Define API endpoints for each functionality

Prompt

ICE PROMPT

Instruction:

Help me build the architecture for an AI defect root-cause analyzer for Azure DevOps (ADO) in a software company. The application pulls closed defects from ADO, has an LLM classify each defect's root cause, SDLC phase and testing gap, aggregates the results, and presents them to QA leadership. Use the available inputs and outputs below.

Use the following input sources to start the questionnaire.

No input is converted to vector embeddings. Defect fields are sent to the LLM directly, in batches, as structured JSON.

Input sources:

Closed defects from the Azure DevOps REST API (WIQL query by work item type, closed/resolved/done state, lookback days, optional area path)

Saved ADO query, given as a pasted query URL (by GUID or folder path) with a personal access token

ADO defect export uploaded by hand (Excel .xlsx / .xls / .xlsm or CSV), for users with no API access

Defect fields: ID, title, description, area path, iteration path, severity, state, resolution / disposition, history / resolution notes, root cause field, tags, created date, closed date

Optional defect fields: comments / discussion, SDLC phase, environment, found-in environment, introduced-in month and year, user impact, parent, work item type

Analysis settings: rejected resolution list, borderline resolution list, review confidence threshold, custom Excel column map

Prompt templates and JSON schemas for the two LLM calls (defect categorization, narrative summary)

Static corrective and preventive action (CAPA) mapping per root-cause category

Output:

Per-defect categorization stored with provenance (model, prompt version, timestamp, input hash):

root cause category, SDLC phase, testing gap flag, one-sentence summary, evidence, confidence score.

Root cause categories: requirements gap, design flaw, coding error, data defect, integration defect, configuration defect, build/deployment defect, test gap, third-party defect, performance defect, security defect, documentation defect, process/communication defect, not a defect, unknown.

SDLC phases: requirements, design, development, testing, build/release, production/operations, not applicable, unknown.

All categories of results: valid, rejected, borderline (e.g. cannot reproduce), needs review (low confidence or unknown), testing gaps, escaped defects (build/release or production), critical/high severity, developer quality vs process error vs unattributed.

Reports in a specific format for the target tool:

Power BI: categorized_defects.csv and categorized_defects.xlsx

Human review: needs_review.csv

Executive narrative: narrative_summary.json (headline, top root causes, hotspot modules, trend note, recommended actions)

Leadership dashboard: KPIs, benchmark meters, top 5 root causes and top 5 areas with CAPA actions, Pareto, root cause vs SDLC phase, monthly trend

System design:

Critical functionalities:

Ingestion pipeline

Categorization pipeline

Functionalities (Critical + optional)

Critical:

Ingestion pipeline -- loads defects from ADO or an export into the database

Step process:

Input (ADO API / saved query / Excel or CSV) --> Normalization (column matching, HTML stripping) --> Source stamping --> Upsert into SQLite by defect ID

Categorization pipeline -- sends defects to the LLM for structured root-cause classification

Step process:

Select uncategorized defects --> Skip defects whose inputs, prompt and model are unchanged

Batching (fixed size, or grouped by area path)

Prompt + JSON schema + Defect batch --> LLM (JSON mode)

Schema validation and lenient field handling (invalid values become "unknown", confidence clamped to 0-1)

Store categorization with provenance

Aggregation -- deterministic statistics computed in code, not by the LLM

Narrative summary -- Prompt + Aggregates --> LLM --> executive narrative

Export -- CSV / Excel for Power BI and the needs-review list

Final defect RCA report / insights generated by the LLM and the aggregation step

Optional:

Streamlit dashboard with setup page and leadership view

Background categorization run that survives a closed browser tab, with live progress

Retry and backoff on rate limits and transient errors, honouring Retry-After

Batch failure isolation (one failed batch does not stop the run)

Token usage and cost tracking per run

Date-window scoping of report and export (since / until on closed date)

API keys kept in the OS credential store, or entered per browser session

Multiple LLM keys with parallel workers

Tech stack:

Backend: Python 3.10+ (pandas, requests, jsonschema), CLI entry point

Frontend: Streamlit (Altair charts)

Database: SQLite (no vector DB)

LLM: Groq (openai/gpt-oss-120b) / Azure AI Foundry, through an OpenAI-compatible chat-completions API

Embedding model: None

Packaging and CI: Python wheel, GitHub Actions (ruff, mypy, pytest)

Deployment server:

Streamlit Community Cloud, or a local install from the wheel or launcher script

Context / background info:

You are the Chief AI Architect and are going to design an architecture for my application with my given inputs.

Persona (role):

You act as a senior architect designing AI agent architecture, with 10+ years of experience.

Output:

Start with the questionnaire and ask me 20 questions with 4 MCQ options each.

Ask one question at a time; once I answer, proceed to the next.

Once I have answered all the questions, produce the final architecture as a

downloadable .md file.

If I don't know an answer, I will ask you to explain. After the explanation I will choose. If I still don't understand, I will ask you to choose the appropriate one, or you can give me a recommendation.

If I feel a question is irrelevant, I will skip it. The architecture must be created based on my answers.

[STRICT] DO NOT HALLUCINATE. ASK ONLY QUESTIONS RELEVANT TO MY REQUIREMENT.

DO NOT GO BEYOND SCOPE.

Tone:

Professional and formal.